

The Rainstorm 4.0.0 Hash Function: Specification (Draft)

Cris (DOSAYGO Corp.), 2023

September 8, 2026

Overview

Rainstorm is a family of keyed hash functions designed by Cris (DOSAYGO Corp.) in 2023. It produces fixed-size outputs of 64, 128, 256, or 512 bits from arbitrary-length byte inputs. Rainstorm uses a 1024-bit internal state and processes input data in 512-bit (64-byte) blocks. Its constants and rotation values were chosen based on extensive testing to ensure desirable mixing properties.

While Rainstorm was created with cryptographic aims, it is currently undergoing evaluation, and no formal proofs of security exist yet. The community is encouraged to analyze and test Rainstorm, applying techniques ranging from heuristic analysis to rigorous cryptography methods.

Domain and Range

Input: A message M is a sequence of bytes ($M[0], \dots, M[L-1]$) with arbitrary length $L \geq 0$.

Seed: A 64-bit unsigned integer s (default $s = 0$). Different seeds produce distinct hash variants.

Output: A digest of length $N \in \{64, 128, 256, 512\}$ bits.

Parameters and Constants

Rainstorm uses fixed 64-bit constants and rotation amounts that were selected through computational searches and empirical testing to encourage strong avalanche properties.

Constants:

$$P = 2^{64} - 1 - 58, \quad Q = 13166748625691186689, \quad R = 1573836600196043749$$

$$S = 1478582680485693857, \quad T = 1584163446043636637, \quad U = 1358537349836140151$$

$$V = 2849285319520710901, \quad W = 2366157163652459183$$

$$K = [P, Q, R, S, T, U, V, W]$$

Rotation constants:

$$Z = [17, 19, 23, 29, 31, 37, 41, 53]$$

Counter values:

$$\text{CTR}_{\text{LEFT}} = 0x\text{fcdab8967452301}_{16}, \quad \text{CTR}_{\text{RIGHT}} = 0x\text{1032547698badcfe}_{16}$$

State Initialization

For a given message length L , seed s , and output length N , the internal state h is a vector of 16 64-bit words.

Define:

$$\begin{aligned} c_0 = 1, \quad c_1 = 2, \quad c_2 = 3, \quad c_3 = 5, \quad c_4 = 7, \quad c_5 = 11, \quad c_6 = 13, \quad c_7 = 17, \\ c_8 = 19, \quad c_9 = 23, \quad c_{10} = 29, \quad c_{11} = 31, \quad c_{12} = 37, \quad c_{13} = 41, \quad c_{14} = 43, \quad c_{15} = 47. \end{aligned}$$

Then:

$$h[i] = s + L + c_i \quad \text{for } i = 0, \dots, 15.$$

Block Processing and Endianness

Rainstorm processes the input in 512-bit (64-byte) chunks. Each chunk is interpreted as eight 64-bit words in little-endian format:

$$data[i] = \text{LE.decode_64}(B[8i : 8i + 7]), \quad i = 0, \dots, 7.$$

The output is produced in little-endian format for each 64-bit word of h .

Rounds and Weak Function

Each 64-byte chunk receives four calls to the weak function, in the order right, left, right, left.

Define $\text{ROTR}(x, n)$ as the 64-bit rotate-right operation on x by n bits.

The Weak Function

$$\text{weakfunc}(h, data, left)$$

If $left = true$:

$$ctr = \text{CTR}_{\text{LEFT}}$$

For $i = 0, \dots, 7$:

$$h[i] \leftarrow h[i] \oplus data[i], \quad h[i] \leftarrow h[i] - K[i], \quad h[i] \leftarrow \text{ROTR}(h[i], Z[i])$$

$$h[i + 8] \leftarrow h[i + 8] \oplus h[i], \quad ctr \leftarrow ctr + h[i], \quad h[i + 1] \leftarrow h[i + 1] - ctr$$

If $left = false$:

$$ctr = \text{CTR}_{\text{RIGHT}}$$

For $i = 8, \dots, 15$:

$$h[i] \leftarrow h[i] \oplus data[i - 8], \quad h[i] \leftarrow h[i] - K[i - 8], \quad h[i] \leftarrow \text{ROTR}(h[i], Z[i - 8])$$

$$h[i - 8] \leftarrow h[i - 8] \oplus h[i], \quad ctr \leftarrow ctr + h[i], \quad h[(i + 1) \bmod 16] \leftarrow h[(i + 1) \bmod 16] - ctr$$

In version 4.0.0, the final right-round subtraction targets $h[0]$, crossing into the other half just as the final left-round subtraction targets $h[8]$. Earlier code wrapped the right subtraction to $h[8]$, causing an exact 64-bit loss of incoming state information for a fixed block. The correction changes digests at every output size. It removes that chosen-state collision construction, but does not establish full-hash cryptographic security.

Padding

After processing all full 64-byte chunks, a final padded block is always formed, including when no bytes remain.

Let r be the remaining length (0 to 63). Initialize a 64-byte array $temp$ with $(0x80 + r) \bmod 256$ in every byte, then overwrite the first r bytes of $temp$ with the remaining data.

The total message length is incorporated in the initial state; no additional length field is appended to this padded block.

Process this final block with 4 rounds of weakfunc, then:

$$\text{for } i = 0 \text{ to } 7 : \quad h[i] \leftarrow h[i] - h[i + 8].$$

The finalization described below is then performed once.

Output and Final Rounds

After processing the final padded block and performing the subtraction step

$$\text{for } i = 0 \text{ to } 7 : \quad h[i] \leftarrow h[i] - h[i + 8],$$

an additional finalization step is applied if $N > 64$ bits. Define:

$$\text{FINAL_ROUNDS} = 2.$$

If the desired output length N is greater than 64 bits, apply $\max(N/64, \text{FINAL_ROUNDS})$ additional calls to

$$\text{weakfunc}(h, temp, \text{true})$$

using the same $temp$ block (the final padded block).

This ensures further mixing of the internal state before extraction of the final digest.

Extracting the Output

The final hash is derived from the first $N/64$ 64-bit words of $h[0 \dots 7]$ in little-endian format:

- $N = 64$ bits: output $h[0]$
- $N = 128$ bits: output $(h[0], h[1])$
- $N = 256$ bits: output $(h[0], h[1], h[2], h[3])$
- $N = 512$ bits: output $(h[0], h[1], h[2], h[3], h[4], h[5], h[6], h[7])$

Each 64-bit word is written to the output buffer in little-endian byte order. No further transformations are performed.

This finalization and extraction process ensures that for each supported output size, the resulting hash is fully defined by the preceding steps, including the additional final rounds for outputs greater than 64 bits.

Empirical Results and Analysis Invitation

Rainstorm’s design choices were guided by empirical testing, including runs through [SMHasher3](#) (by Frank J. T. Wojcik). The promoted version 4.0.0 implementation passed all 253 checks in the full extended 256-bit native campaign, including BadSeeds. That suite does not instantiate 512-bit outputs. On the tested ARM cloud machine, the small-key measurement was 249.64 cycles per hash. Performance and passing statistics are not security proofs.

These observations are encouraging but are not conclusive cryptographic proofs. Rainstorm has not been formally analyzed by the broader cryptographic community. Interested researchers and practitioners, whether new to cryptanalysis or established in the field, are invited to scrutinize Rainstorm’s construction, test it against known cryptographic attacks, and share their findings.

Test Vectors

Below are sample test vectors (with $s = 0$ and $N = 256$):

Message: ""
Digest: bf6aa062a4c6ccf8b69697494100743f24da78e0e0140af278f3156772734b49

```
Message: "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"  
Digest: a3eeff6dbbf8213dd3b3b00a7ad813a76dc20f9c8ddb4424ee906bca3616e7b9
```

Message: "The quick brown fox jumps over the lazy dog"
Digest: 2343e4baabecf8be423ab643fcdfa113e14bf75e7a5f8a6a37f02a260282ac45

Message: "The quick brown fox jumps over the lazy cog"
Digest: 336f151a152a50930cf47ac8a787f55bc37e711945cc1845ec1aa4c4fdb257d4

Message: "The quick brown fox jumps over the lazy dog."
Digest: 638cce058cae9895f61c5c03a6d75e8c56a8d4a138a01d6bf60038d076e12177

Message: "After the rainstorm comes the rainbow."
Digest: 3ca3ed36a2503b5908d5deb596951a57b7dad37fc7d2dda756c91124acafdfd1

Message: "aaa"
Digest: 3cb0c1d4fd07567910057bf9a884651da54ae2117619b3c143e96870032eb5e3

Conclusion

This document provides a formal specification of Rainstorm’s design, constants, and processing steps, as well as initial empirical observations. The cryptographic community is invited to analyze Rainstorm for properties such as collision resistance, preimage resistance, and differential security. Only through public scrutiny and rigorous testing can Rainstorm be evaluated as a potential candidate for widespread cryptographic use.